

Work Sheet 2 (Data Structures)

1. The **Sports Week** at your university is just around the corner, and as the **sports coordinator**, it's your responsibility to manage the **list of players** for each department. As the event draws nearer, players will be **added** and **removed** from the list dynamically. Your task is to ensure that the **player list** is maintained efficiently, and you must handle the changes in the roster, ensuring there's enough space in the list as new players join or leave.

To achieve this, you'll use a **List ADT** that not only allows you to **add** and **remove** players, but also supports several important functions, such as **resizing the list** when it becomes full, **finding players** in the list, checking if the list is **empty** or **full**, and printing the current list of players.

Key Operations of the List ADT:

Add Player: Insert a new player in the list.

Remove Player: Remove a player who leaves the team.

Find Player: Search for a specific player in the list.

Check if the List is Full or Empty: Use `isFull()` and `isEmpty()` to check the status of the list.

Resize List: Automatically increase the capacity of the list when it's full using `resize()`.

Print Players: Display the current list of players for each department.

NOTE: The C++ program use Array-Based List with Free List for underlying Data Structure.

2. From the following set of operations create a list that has forward insert operation.

Index	Data	Next
-------	------	------

0	*	*
1	*	*
2	*	*
3	*	*
4	*	*

Operations List:

1. **Forward Insertion of 10**
2. **Forward Insertion of 20**
3. **Forward Insertion of 30**
4. **Delete of 10**
5. **Delete of 30**
6. **Forward Insertion of 10**

Note: You need to draw list state after each operation.

3. From the following set of operations create a list that has back insert operation.

Index	Data	Next
-------	------	------

0	*	*
1	*	*
2	*	*
3	*	*
4	*	*

Operations List:

1. **Backward Insertion of 10**
2. **Backward Insertion of 20**
3. **Backward Insertion of 30**
4. **Delete of 10**
5. **Delete of 30**
6. **Backward Insertion of 10**

Note: You need to draw list state after each operation.

4. From the following set of operations create a list that can manage both **Backward** and **Forward** insert operation.

Index	Data	Next
-------	------	------

0	*	*
1	*	*
2	*	*
3	*	*
4	*	*

Operations List:

1. **Backward Insertion of 10**
2. **Backward Insertion of 20**
3. **Forward Insertion of 30**
4. **Forward Insertion of 40**
5. **Backward Insertion of 10**
6. **Delete of 10**

Note: You need to draw list state after each operation. **Conclude your point that managing both operations on single list is possible OR not possible?**